

Feature Detail & Demo Reference

This document is the **single source of truth** for implementing the CS2 RCON web interface (this repo is a fork of [nokit](https://github.com/codevski/nokit) (<https://github.com/codevski/nokit>) / defuse). It captures everything observed in the public demo at <https://nokit.app/demo> (<https://nokit.app/demo>) so that developers **never need to visit the demo again**.

It is split into two parts:

1. **Feature Status Table** — a quick overview of what is actually implemented vs. what is currently only a static demo / planned.
2. **Detailed Feature Specifications** — an exhaustive, per-feature reference of every UI element, data field, column, interaction, chart, color code, status indicator, example value, and technical/implementation note recorded from the demo.

Demo versions observed: `nokit v0.1.0-rc.1` (footer), `relay v0.4.2` (sidebar footer). License MIT.
Deploy target: Docker image `ghcr.io/nokit/nokit:latest`, default web port `8080`, data volume `/data`, single ~14 MB Go binary running alongside `srcds` over loopback RCON.

Implementation will begin with the **Dashboard**.

Section 1 — Feature Status Table

Feature	Status	Notes
Login / Auth	✓ Implemented	Password-based login, session tokens in SQLite
Multi-server switcher	✓ Implemented	Add/remove servers (name, RCON host, password) via header
RCON Console	✓ Implemented	Full terminal UI: timestamps, color-coded output, ↑↓ history, Ctrl+L clear, inline suggestion list autocomplete (top 8 ranked hits, prefix-highlighted, auto-opens after first character, ↑↓/Tab/Enter/mouse selection, Escape to dismiss) for 5000+ CS2 CVARs/commands, RCON macros sidebar with local-storage persistence, clickable history panel, live/paused scroll toggle, Copy session & Export
Dashboard	✓ Implemented	Live stat cards (CPU/tick/players) with sparklines, server status, quick actions, round info, recent output — polls <code>status / stats</code> over RCON every 6s
Live Logs	✓ Implemented	Real-time server console ingested over UDP (CS2 <code>logaddress_add , internal/loghub</code>) and HTTP (CS2 <code>logaddress_add_http</code> → <code>POST /api/logs/http</code>) and streamed over SSE (<code>GET /api/logs/stream</code>); configurable line retention (default 500, 50–2000), auto-scroll with manual-scroll pause, Clear, Download <code>.log</code> , connection status indicator. Bind port <code>CS2_LOG_LISTEN_PORT</code> (default <code>27500</code>), sink

Feature	Status	Notes
		CS2_LOG_SINK_ADDR . No Docker socket required.
HTTP Log Listener	✓ Implemented	Public POST /api/logs/http endpoint accepts CS2 logaddress_add_http plain-text payloads (loghub.ParseHTTPBody) and publishes each line into the same internal/loghub hub as the UDP listener via Hub.Publish , so HTTP-sourced logs appear in Live Logs and drive the same workshop-map download verification. Coexists with the UDP listener (both run simultaneously). Optional CS2_LOG_HTTP_SINK_URL RCON auto-config and CS2_LOG_HTTP_TOKEN shared-secret guard (?token= / X-Log-Token).
Players	✗ Not built	Demo only / planned
Maps	✓ Implemented	Standard map pool (12 maps), favorites system (localStorage), two-mode workshop map management (internal/workshop) — Mode A (default) lists installed maps via RCON ds_workshop_listmaps (fallback maps *) → GET /api/servers/{id}/maps/workshop , resolving workshop IDs from a DB cache (workshop_maps) populated when maps are loaded through the panel; Mode B (opt-in, when a volume is mounted at /workshop/<serverId>) scans the filesystem for exact ID↔name mapping incl. multiple versions, with runtime write-

Feature	Status	Notes
		probed uninstall (DELETE / api/servers/{id}/maps/workshop/{id}). Load endpoint POST /api/servers/{id}/maps/workshop/load . Workshop map thumbnails via the Steam Web API (internal/steam , cached on disk, served from GET /api/maps/thumbnail/{id} , enabled by STEAM_API_KEY), map cycle editor, RCON integration (changelevel, host_workshop_map)
CVAR Presets	✗ Not built	Demo only / planned
Config Editor	✓ Implemented	File browser tree view, code editor with line numbers, unsaved changes tracking, save/reload/exec via RCON, support for .cfg and .json files
Plugins	✗ Not built	Demo only / planned
Scheduler	✗ Not built	Demo only / planned
Admin	✗ Not built	Demo only / planned

Section 2 — Detailed Feature Specifications

Global Layout & Chrome (present on all pages)

These elements are shared across every page and should be built as part of the app shell.

Top navigation bar (full width, dark)

Left cluster:

- **nokit** logo (green square “n” icon + wordmark) — top-left.
- **Server switcher dropdown**: label `fra1.mm` , subtext `5.39.42.118:27015` , with a small green online dot and a chevron (dropdown to switch servers / add-remove servers). See §11.
- **tickrate** indicator: `127.8`
- **players** indicator: `9/12`
- **map** indicator: `de_mirage`

Right cluster:

- **Search box**: placeholder `cvars, players...` with a `⌘K` / `K` keyboard-shortcut hint badge (command

palette style search). See §13.

- Small **HK** / **⌘K** badge button.
- **External-link icon** (open server / launch).
- **Settings gear icon**.
- **User menu**: avatar + `m0use@` (current user), with dropdown.
- **Power / logout icon** (far right).

Left sidebar navigation (grouped)

- **SERVER**
 - Dashboard (home icon) — active/highlighted green
 - RCON Console — has a small **⌘K** / **HK** badge on the right
 - Live Logs
 - Players
- **CONFIGURATION**
 - Maps
 - CVAR Presets
 - Config Editor
- **SYSTEM**
 - Plugins — has an amber/orange numeric badge **1** (e.g. update/notification count)
 - Scheduler
 - Admin

Sidebar footer: `relay v0.4.2`, a throughput indicator `↑ 2.1 kb/s · ↓ 18 kb/s`, and a green **● WS** (WebSocket connected) status.

Page footer (bottom, full width)

- Left: `nokit v0.1.0-rc.1 · MIT · ★ github · docs · changelog`
- Right (status strip): `● srcds OK · ● relay OK · 10/07/2026 · 16:16:14 · Europe/Berlin (UTC+2)`
— shows service health, date, live clock, timezone. The footer clock ticks live (observed 16:16 → 16:30 across the session).

1. Dashboard

Page title: **Dashboard**. Subtitle: `fra1.mm · 5.39.42.118:27015 · connected via rcon`.

Top-right of page: green **● live** pill + **↻ Refresh** button.

Top stat cards (4 across)

Card	Big value	Sub-line
PLAYERS	<code>9 / 12</code> (green <code>+2</code> delta badge)	<code>last 5m</code>
TICK RATE	<code>127.8 hz</code>	<code>target 128</code>
CPU	<code>22 %</code>	<code>4-core · 0.88 load avg</code>
RAM	<code>2.1 / 8 GB</code>	<code>resident</code>

Current round panel (left, ~2/3 width)

Header: **Current round** — right side: tick #88669 · uptime 11h 02m .

- Map thumbnail image with label `de_mirage` and a `competitive` badge overlay.
- Field rows (label → value):
- map → `de_mirage` (active pool)
- gamemode → `competitive` · MR12 + OT
- round → `8 / 24` – CT 4 : 3 T
- round time → `0:47` remaining
- warmup → `ended` · `live`
- session → `2h 14m` · matchzy "live"

Quick actions panel (right, ~1/3 width)

Header: **Quick actions**. Buttons top-to-bottom:

-  Restart server
- Segmented toggle: [`match` | `practice` | `pug`] — **match** selected (highlighted).
-  No password
-  End warmup
-  Auto-shuffle teams
-  Stop server (red/danger styling)

Mini metric chart widgets (4 across, sparkline/area charts)

Widget	Range	Value	Chart color	Footer
Players	24h	9 / 12	green line	peak 12 · avg 6.4
CPU	5m	22%	blue area	peak 41% · sustained 28%
Memory	5m	2.1 GB	purple line	26% of 8 GB
Tick	5m	127.8 hz	orange line	target 128 · jitter 0.4

stats // rcon dump panel (bottom-left)

Header:  **stats // rcon dump** — right: refreshed 4s ago .

Tabular columns: CPU | NetIn | NetOut | Uptime | Maps | FPS | Players | Svms | +-ms | ~tick

Row values: 22.0 | 18.4 | 61.7 | 11h02m | 14 | 127.8 | 9 | 7.42 | 0.40 | 127.8

Below the table (key/value lines):

- version: 1.40.6.3 (cs2)
- os: linux x86_64 (debian 12)
- gotv: 1x spec on :27020
- protocol: v15 / build 9462
- vac: secure
- workshop: 16 maps · 281 MB

Recent events panel (bottom-right)

Header:  **Recent events** — right: view all → link.

Event rows (timestamp + actor + action):

- 12:51 m0use@ ended warmup
- 12:49 jaeger kicked h4rddy_woods - afk
- 12:40 m0use@ changed map: de_mirage
- 12:38 jaeger applied preset: MR12 (24+0T)
- 12:35 m0use@ edited gamemode_competitive.cfg
- 12:18 system GotvFix: bind failure

Implementation notes: dashboard/stats auto-refresh every ~4s. The stat cards and mini charts are driven by the `status` RCON dump plus a metrics history buffer. The Recent events panel mirrors the Admin → Audit log.

✓ **Current implementation status (this fork):** The Dashboard is built and is the default view when a server is selected (`web/src/pages/dashboard-page.tsx`), with a Dashboard/Console tab bar in `server-page.tsx` . It polls the `status` and `stats` RCON commands every 6s via the `useServerStats` hook (`web/src/hooks/useServerStats.ts`), parsing them with `web/src/lib/rcon-parse.ts` (tolerant of multiple CS2 output formats).

- **Stat cards** (`StatsCard` + `Sparkline`): Players (green), Tick rate (orange, target 128), CPU (blue area chart, with FPS sub-line), and RAM. Sparklines are rendered from a rolling in-memory history. RAM is shown as `n/a` because CS2's `stats / status` do not report server memory over RCON.
- **Server status** (`ServerStatus`): online/offline dot, name, address, map, hostname, players, uptime, version, OS — all parsed from `status / stats` .
- **Quick actions** (`QuickActions`): Restart server (`_restart` , confirmed), match/practice/pug segmented toggle (`exec <mode>`), clear password (`sv_password ""`), End warmup (`mp_warmup_end`), Auto-shuffle teams (`mp_scrambleteams 1`), and Stop server (`quit` , confirmed) — each sends the command over the existing `/api/servers/:id/rcon` endpoint and surfaces a success/error line.
- **Round info** (`RoundInfo`): shows map, phase (live/offline) and player count from `status` ; round number/score/timer are shown as `-` with a note that they will come from the Live Logs stream (not exposed by `status / stats`).
- **Recent output** (`RecentOutput`): shows the last lines of the most recent `status` poll and a link that switches to the full Console tab.

2. RCON Console

Page title: **RCON Console**. Subtitle: `tab to autocomplete · ↕ history · ctrl-l clears` .

Top-right controls: `[ live | paused ]` segmented toggle (live selected), `📄 Copy session` button, `⌵ Export` button.

Console output area (left, large monospace terminal, ~2/3 width)

Green/amber terminal text. Commands echoed with timestamp prefix `rcon HH:MM:SS >` . Command values are syntax-highlighted (numbers/limits in color). Example session content:

- `rcon 12:50:00 > status`
- `hostname: nokit | fra1.mm · map: de_mirage · players: 10/12 · pubkey: 8f12...ed94`
- `rcon 12:52:02 > mp_warmuptime 15`
- `"mp_warmuptime" = "15" ( def. "30" min. 0.000000 max. 90.000000 ) replicated - Time after a new round...`
- `rcon 12:54:04 > sv_password`
- `"sv_password" = "*****" archive notify - Password to join the server.`
- `rcon 12:56:06 > mp_warmup_end`

- [matchzy] warmup ended. live in 5...
- - - live - -

Input bar (bottom of console)

Prompt: `rcon`) monospace input field, pre-filled demo text `mp_war` . Has a small submit/enter affordance on the right.

Quirk: in the demo the input is static — typing appends but no live autocomplete dropdown fires. The subtitle advertises tab-to-autocomplete, ↑ ↓ history, and ctrl-I to clear as intended behaviors.

RCON macros panel (right sidebar)

Header: ⚡ **RCON macros**. Each macro is a button showing a friendly label + the underlying command (greyed monospace):

- | Label | Command |
|-------------------|--------------------------------------|
| ----- ----- | |
| warmup → live | <code>mp_warmuptime 15 ; m...</code> |
| reset score | <code>mp_restartgame 1</code> |
| knife → side pick | <code>matchzy_knife</code> |
| pause match | <code>matchzy_pause</code> |
| kick all bots | <code>bot_kick</code> |
| demo: start | <code>tv_record demos/manu...</code> |
| demo: stop | <code>tv_stoprecord</code> |
| fix stuck round | <code>mp_unpause_match ; m...</code> |
- + New macro... button at bottom (create custom macro).

History panel (right sidebar, below macros)

Header: 🏹 **History**. Clickable recent commands list (re-run on click):

- status
- mp_warmuptime 15
- sv_password
- mp_warmup_end
- changelevel de_inferno
- tv_record demos/2026-05-21

✓ **Current implementation status (this fork):** The RCON Console is fully built as a real terminal UI. The component lives in `web/src/components/console.tsx` (exported as `Console`) and is rendered from the Console tab in `web/src/pages/server-page.tsx`. It uses the existing `POST /api/servers/:id/rcon` endpoint — no new backend routes were added.

- **Layout:** two-column split — a left terminal area (~65% width) and a right sidebar (~35% width).

- **Terminal output:** every command is echoed with a timestamp prefix `rcon HH:MM:SS >` and output is color-coded — green for the command prompt/echo, amber for CVAR values, red for errors, and white/muted for regular output.

- **Input bar:** monospace prompt `rcon >` with shell-style keyboard shortcuts — `↑↓` navigate command history (when suggestion list is not visible), `Ctrl+L` clears the terminal. Subtitle updated to `type to suggest · ↑↓ navigate · tab/enter to fill · ctrl-l clears`.

- **Autocomplete suggestion list:** A floating list appears automatically after the **first character** is typed (no key press required). Suggestions are drawn from **5000+ CS2 CVARs/commands** (`web/src/data/cs2-commands.json`, sourced from [ArminC-CS2-Cvars](https://github.com/armync/ArminC-CS2-Cvars) (<https://github.com/armync/ArminC-CS2-Cvars>)) and ranked in two tiers — prefix matches first, then contains matches — capped at **8 best hits** for clarity. The matched portion of each suggestion is **bold and highlighted** (primary colour, or underlined when that item is selected). The list has a header showing the hit count and a footer showing keyboard hint text (`↑↓ navigate · tab/enter to fill · esc to close`). Selection works via: `↑↓` arrow keys to move the highlight (input value is not replaced until confirmed); **Tab** to fill the input with the selected item (or the first hit if nothing is highlighted); **Enter** to fill the input if an item is highlighted; **mouse click** on any item; or **Escape** to dismiss (the list re-opens on the next keystroke). When the suggestion list is visible, `↑↓` navigate it; when it is dismissed or empty, `↑↓` navigate command history as usual.

- **Live/paused scroll toggle:** in live mode the view auto-snaps to the newest line; paused mode lets the user scroll freely through the backlog without being pulled back to the bottom.

- **Session tools:**  Copy session copies the full session log to the clipboard, and  Export downloads it as a `.txt` file.

- **RCON macros panel:** 8 built-in macros (warmup → live, reset score, knife round, pause match, kick bots, demo start, demo stop, fix stuck round) plus custom macro creation via the `+ New macro` form. All custom macros are persisted in `localStorage` under the key `nokit_console_macros`.

- **History panel:** shows the last 20 unique commands (most recent first); clicking an entry pastes it into the input. History is persisted across reloads in `localStorage` under the key `nokit_console_history`.

3. Live Logs

Page title: **Live Logs**. Subtitle: `srcds_logaddress · SSE to nokit-relay · 19 retained · 19 shown` (shown count updates when filtering).

Top-right controls: green  **LIVE** pill,  **Pause** button,  **Download .log** button.

Filter tabs (with count badges)

All 19 · Chat 3 · Kills 4 · Rounds 5 · Connect / team 2 · Plugin 1

- **Functional:** clicking a tab filters the stream by event category and updates the “N shown” count (verified — Chat tab showed only the 3 chat lines).

Search / grep bar

- Input with placeholder: `grep – e.g. STEAM_1:0, "awp", defused` and a `/` regex toggle affordance at the right.
- Toggle checkbox: **hide rcon noise**.

- Far right: `tail -f` · 1.4k lines/min throughput label.

Log stream (monospace, color-coded by event type)

Each line: `HH:MM:SS <TYPE> message`. The event-type keyword is colored. Types seen: `rcon`, `world`, `round`, `kill`, `chat`, `conn`, `team`, `srv`, `plug`.

Example lines:

```
- 12:51:08 rcon    > mp_warmup_end
- 12:51:08 world  World triggered "Warmup_End"
- 12:51:09 round  World triggered "Round_Start"
-
12:51:14 kill    "m0use<2><STEAM_1:0:48211904><CT>" killed "zywoo<7><STEAM_1:1:11923844><T>"
with "ak47" (headshot)
- 12:51:23 chat    "m0use<2><STEAM_1:0:48211904><CT>" say "gg wp"
- 12:51:25 kill    "NEK0<5><STEAM_1:1:78231004><T>" killed "ferr<3><STEAM_1:1:91134472><CT>"
with "awp"
- 12:51:27 round  Team "T" triggered "SFUI_Notice_Target_Saved" (CT "1") (T "0")
- 12:51:27 round  World triggered "Round_End"
- 12:51:32 conn    "jL<9><STEAM_1:0:90120431>>" connected, address "78.34.221.41:27015"
- 12:51:34 team    "jL<9><STEAM_1:0:90120431><Unassigned>" switched from team <Unassigned> to
<TERRORIST>
- 12:51:40 chat    "NEK0<5><STEAM_1:1:78231004><T>" say "rotate B"
- 12:51:55 rcon    > status
- 12:51:55 srv     hostname: nokit | fra1.mm · map: de_mirage · players: 10/12
- 12:52:01 kill    "twist<6><STEAM_1:1:12349803><CT>" killed "bymas<8><STEAM_1:1:43712001><T>"
with "deagle" (headshot)
- 12:52:08 chat    "olof<4><STEAM_1:0:55102941><CT>" say_team "info one toxic at long"
-
12:52:12 plug     [WeaponPaints] failed to fetch skin metadata for STEAM_1:1:43712001 (timeout
5s)
-
12:52:30 kill    "zywoo<7><STEAM_1:1:11923844><T>" killed "m0use<2><STEAM_1:0:48211904><CT>"
with "awp" (penetrated)
- 12:52:55 round  Team "CT" triggered "SFUI_Notice_Bomb_Defused" (CT "1") (T "1")
- 12:53:01 rcon    > changelevel de_inferno
```

Stream footer

`tail`) awaiting next event · backpressure 0 · last frame 1.2s ago with a green ● **SSE** connection indicator (right).

Implementation notes (demo/reference): logs are ingested via `srcds_logaddress` and streamed to the browser over **Server-Sent Events (SSE)** through nokit-relay. The stream shows backpressure and last-frame timing.

✓ As implemented in this fork

The shipped Live Logs panel follows the same approach as the demo spec (`srcds_logaddress`) but keeps the ingestion in-process — no external relay or Docker socket required:

- **Source / ingestion (internal/loghub):** the backend binds a **UDP** socket (`CS2_LOG_LISTEN_PORT`, default 27500) and receives the CS2 server's logs directly. CS2 is told where to send them via RCON on startup

(`logaddress_add <CS2_LOG_SINK_ADDR>`; `log on` , retried until the server is reachable). Each incoming datagram is a Source “log packet” (`0xFFFFFFFF` header, `R / S` type byte, optional `sv_logsecret` , then the `L ... body`); `loghub.ParsePacket` strips the framing and yields a clean line.

- **Fan-out:** a small pub/sub hub broadcasts every parsed line to all current subscribers over buffered channels. A slow subscriber drops lines rather than blocking the UDP reader.
- **Transport:** each line is relayed to the browser over **SSE** as a `data: <line>\n\n` event via `GET /api/logs/stream` (session-cookie protected, same auth as the rest of `/api/`). A `: heartbeat` comment is sent every 15s to keep the connection and any intermediary proxies alive.
- **Configuration:**
 - `CS2_LOG_LISTEN_PORT` — UDP port the hub binds (default `27500`).
 - `CS2_LOG_SINK_ADDR` — address handed to the CS2 server via `logaddress_add` , as reachable from the CS2 container (e.g. `defuse:27500`). If empty, auto-config is skipped and the operator wires `logaddress_add` themselves.
- **Lifecycle / cleanup:** each SSE client subscribes to the hub on connect and is unsubscribed (its channel closed) when the request context is cancelled (client disconnect). On hub shutdown the handler emits an `event: end` frame and the browser shows `reconnecting` .
- **Frontend (`web/src/components/logs.tsx`):** connects with `EventSource` , renders lines with best-effort color coding, and provides:
 - a **Max lines** numeric input (default `500` , range `50 – 2000` , persisted to `localStorage`) capping how many lines are retained in memory/DOM;
 - **auto-scroll** to the bottom that **pauses** automatically when the user scrolls up, with a jump to latest pill to resume;
 - a **Clear** button and a **Download .log** export;
 - a **connection status indicator** (`connecting` / `connected` / `reconnecting` / `error`).
- **Deployment:** the panel publishes `27500/udp` ; no `docker-cli` and no Docker socket mount are needed. The bundled `docker-compose.yml` runs the panel and a `joedwards32/cs2` server on a shared network and sets `CS2_LOG_SINK_ADDR=defuse:27500` .

✓ HTTP log listener (`logaddress_add_http`)

Newer CS2 builds (and some network setups where UDP is impractical) deliver logs over **HTTP** using `logaddress_add_http <url>` , which POSTs each log line as a plain-text HTTP body. This fork supports that transport **alongside** the UDP listener — both run at the same time and feed the **same** in-process hub, so the source (UDP vs HTTP) is transparent to the rest of the panel:

- **Endpoint (`internal/api/logs.go` , `LogsIngestHTTPHandler`):** `POST /api/logs/http` . Registered on the **public** mux (like `/api/login` and `/api/health`) because a game server cannot present a panel session cookie. It reads the POST body (capped at 1 MiB), parses it with `loghub.ParseHTTPBody` , and publishes each resulting line via `Hub.Publish` . It always replies `200 OK` with an empty body (CS2 ignores the response).

- **Parsing (`loghub.ParseHTTPBody`):** unlike the UDP transport, HTTP delivery carries no `0xFFFFFFFF` header or `R / S` type byte — the body is one or more log lines, each typically prefixed with the Source `L` marker and terminated by a newline. Each line is normalised with the shared `cleanLine` helper (the same trimming `ParsePacket` uses), so HTTP and UDP lines render identically.
- **Shared pipeline (`Hub.Publish`):** published lines flow through the exact same fan-out (`broadcast`) as UDP lines, so they appear in the Live Logs SSE stream (`GET /api/logs/stream`) and drive the **same workshop-map download verification** — the messages emitted after `host_workshop_map <id>` are shown and colour-coded in the live view regardless of transport. `Publish` is a no-op once the hub is closed.
- **Auto-configuration (`cmd/defuse/main.go`):** when `CS2_LOG_HTTP_SINK_URL` is set, the panel appends `logaddress_add_http <url>` to the RCON startup sequence (alongside the UDP `logaddress_add`), so both sinks are registered automatically. Either sink (UDP, HTTP, or both) triggers auto-config.
- **Security:** the endpoint is unauthenticated by default. Setting `CS2_LOG_HTTP_TOKEN` makes it require a matching token via a `?token=<secret>` query parameter or an `X-Log-Token` header; mismatches get `401` .
- **Configuration:**
 - `CS2_LOG_HTTP_SINK_URL` — full endpoint URL as reachable from the CS2 container (e.g. `http://defuse:8080/api/logs/http`). Empty $\Rightarrow$ operator wires `logaddress_add_http` themselves.
 - `CS2_LOG_HTTP_TOKEN` — optional shared secret guarding the endpoint.
- **Tests:** `TestParseHTTPBody` / `TestPublish` (`loghub`) and `TestLogsIngestHTTPHandler` / `TestLogsIngestHTTPHandlerToken` / `TestLogsIngestHTTPHandlerNilHub` (`api`) cover body parsing, hub fan-out, the token guard, and the nil-hub path.

Filter tabs, grep/regex search, and the throughput/backpressure footer from the demo spec above are **not yet** wired up — the current panel focuses on a reliable real-time stream with retention, auto-scroll, and export.

4. Players

Page title: **Players**. Subtitle: `auto-refresh every 4s · 10 online · 4 on watchlist · 4 bans` .

Top-right buttons: `Auto-shuffle` , `Mass kick...` , green `+ Add to watchlist` .

Tabs (with count badges)

`Live 10` · `Watchlist 4` · `Bans 4` · `Session history`

Common controls

- Search input: placeholder `search name, steamid, country, notes` .
- Team filter segmented toggle: `[ all | CT | T | spec ]` (all selected).
- `Filter` button (right).

Live tab — table columns

`NAME | STEAMID | TEAM | PING | TIME | K/D | COUNTRY`

- **NAME:** some rows have an amber flag icon + a green `watch` badge (on watchlist). Rows with high ping shown in a warning color.

- **TEAM:** colored pill — CT (blue) / T (amber/orange).
- **PING:** e.g. 18ms (high ping like 86ms highlighted).
- **TIME:** connected time e.g. 1:24:11 .
- **K/D:** ratio e.g. 1.31 .
- **COUNTRY:** flag icon + 2-letter code.

NAME	STEAMID	TEAM	PING	TIME	K/D	COUNTRY
m0use_ (watch)	STEAM_1:0 :48211904	CT	18ms	1:24:11	1.31	SE
ferr	STEAM_1:1 :91134472	CT	22ms	1:24:08	0.94	DE
kqly.exe	STEAM_1:0 :32220110	CT	41ms	1:18:54	1.07	PL
twist	STEAM_1:1 :12349803	CT	24ms	1:17:02	1.18	SE
olof	STEAM_1:0 :55102941	CT	28ms	1:11:33	0.88	SE
NEKO	STEAM_1:1 :78231004	T	19ms	1:24:11	1.42	JP
zywoo*	STEAM_1:0 :11923844	T	26ms	1:23:01	1.95	FR
bymas	STEAM_1:1 :43712001	T	33ms	0:58:21	1.04	LT
jL	STEAM_1:0 :90120431	T	29ms	0:42:11	1.08	FI
h4rdy_woo ds (watch)	STEAM_1:1 :77124930	T	86ms	0:11:54	0.73	BR

Row hover/click reveals inline action buttons (right side of row): `profile` , `spectate` , `kick` , `ban...` (red/danger).

Watchlist tab — table columns

NAME | STEAMID | REASON | STATUS
NAME	STEAMID	REASON	STATUS
m0use_	STEAM_1:0:48211904	wallhack reports x4	● online · this server (green)
h4rdy_woods	STEAM_1:1:77124930	griefing — voted twice	● online · this server (green)
silent.knife	STEAM_1:1:20300912	team-damage spammer	● offline (grey)
badboy_3000	STEAM_1:0:65332100	racism — 1mo warn	● offline (grey)
 - Each row has an amber flag icon. STATUS is a colored pill (green online / grey offline).

Bans tab — table columns

```

NAME | STEAMID | REASON | BY | EXPIRES
| NAME | STEAMID | REASON | BY | EXPIRES |
|-----|-----|-----|----|-----|
| lobotomy | STEAM_1:0:55512090 | wallhack — demo evidence | m0use@ | 2026-06-12 18:00 (amber)
|
| rage_quitter | STEAM_1:1:11203044 | griefing | admin | 2026-05-24 04:00 (amber) |
| idiot | STEAM_1:0:99687700 | racism — auto-detect | system | permanent (red) |
| spinbot | STEAM_1:1:00091123 | aim hack — confirmed demo | m0use@ | permanent (red) |
- EXPIRES: temporary bans show a date pill (amber); permanent bans show a red permanent pill.

```

Session history tab

Empty state message: No session history pruned yet. Last 30d retained.

Implementation notes: player list auto-refreshes every ~4s (parsed from `status`). Watchlist and bans are persisted server-side.

5. Maps

Page title: **Maps**. Subtitle: 12 standard · 4 workshop · current: `de_mirage`.

Top-right buttons: Sync workshop, Browse collection.

FAVORITES row

Section label `FAVORITES` with a show favs only toggle (right).

Compact favorite cards (thumbnail + name + mode badge):

- Mirage (`active` badge) · comp
- Inferno · comp
- Dust II · comp
- Overpass · comp
- aim_botz · practice

STANDARD POOL grid (4-across cards)

Each card: map thumbnail (radar-style with A/B bombbsite markers), map name, a favorite star, and a mode badge (`comp`, `hostage`, `practice`). The active map has a green `active` badge and highlighted green border.

Maps: `de_mirage` (active), `de_inferno`, `de_dust2`, `de_nuke`, `de_overpass`, `de_anubis`, `de_vertigo`, `de_ancient`, `cs_office` (hostage), `cs_italy` (hostage), `de_train` (comp), `aim_botz` (practice).

Map change interaction (verified): clicking a map card selects it as active — the green `active` badge and highlight move to the clicked card (both its pool card and favorites card update). This is how you change the current map.

WORKSHOP section

Label: `WORKSHOP` 281 MB CACHED · `/HOME/CS2/SERVER/CSGO/MAPS/WORKSHOP`.

- Workshop ID input: placeholder workshop ID — e.g. 3070900859.
- `host_workshop_map` button.
- Green `Download & switch` button.

Workshop map cards (thumbnail + name + author + subscriber count + workshop ID +):

```

| Name | Author | Subs | Workshop ID |

```

```
|-----|-----|-----|-----|
| de_cache_redux | by shawn | 281k subs | #3070900859 |
| awp_lego | by 3klikesphilip | 92k subs | #3070192884 |
| surf_kitsune_ksf | by kitsune | 14k subs | #3071182234 |
| de_season | by sumamon | 441k subs | #3072201991 |
```

MAP CYCLE editor (bottom)

Label `MAP CYCLE` . Ordered chip list (each chip has a numeric order + name + `x` remove):

```
order: 01 de_mirage x · 02 de_inferno x · 03 de_nuke x · 04 de_dust2 x · 05 de_anubis x · 06
de_overpass x · 07 de_ancient x
```

- `+ add` button to append a map.
- Right: `save mapcycle.txt` button (writes the cycle to mapcycle.txt).

- ✓ **Current implementation status (this fork):** The Maps panel is fully built as a functional page. The component lives in `web/src/pages/maps-page.tsx` (exported as `MapsPage`) and is rendered from the `/servers/:id/maps` route in `App.tsx`. It uses the existing `POST /api/servers/:id/rcon` endpoint for map changes — no new backend routes were added.
 - **Standard map pool:** 12 CS2 maps loaded from `web/src/data/cs2-maps.json` (Mirage, Inferno, Dust II, Nuke, Overpass, Anubis, Vertigo, Ancient, Train, Office, Italy, aim_botz) with categorization by mode (comp, hostage, practice).
 - **Favorites system:** Users can star/unstar maps; favorites are persisted in `localStorage` under `nokit_map_favorites`. A “show favs only” toggle filters the grid to show only favorited maps. Favorite maps are displayed in a compact horizontal row at the top for quick access.
 - **Active map tracking:** The currently active map is highlighted with a green border and “● active” badge. Map changes are triggered via RCON `changelevel <mapid>` command.
 - **Workshop maps (two-mode management):** Installed workshop maps are listed through `GET /api/servers/:id/maps/workshop`, which returns `{mode, writable, maps[]}`. The backend logic lives in the `internal/workshop` package (`Resolve()` picks the provider per server at request time):
 - **Mode A — RCON + DB cache (default):** lists installed maps via RCON `ds_workshop_listmaps` (falling back to `maps *`), which returns bare map **names** only. Workshop IDs are resolved from the SQLite `workshop_maps` cache, populated when a map is loaded through the panel (`POST /api/servers/:id/maps/workshop/load` runs `host_workshop_map <id>` and, after confirming the loaded name from `status`, upserts the `id → name` mapping). Maps with a cached ID show a thumbnail + `instant` badge; maps without one show a clean `no id` card and are switched by name via `ds_workshop_changelevel <name>`. Cannot disambiguate multiple versions sharing a name.
 - **Mode B — filesystem scan (opt-in):** activated automatically when a volume is mounted at `/workshop/<serverId>` (base `WORKSHOP_BASE`, default `/workshop`; per-server override `WORKSHOP_PATH-<UPPER_ID>`). Scans numeric `<workshopId>/*.vpk` subfolders for an exact ID↔name mapping including multiple versions grouped per card (`scanned` badge). **Uninstall** (`DELETE /api/servers/:id/maps/workshop/{id}`) is offered only in Mode B and only when the mount is writable — writability is auto-detected at runtime by a write-probe (create+delete a temp file), never a config flag. Mount `:ro` to keep uninstall disabled.
 - **Workshop map thumbnails (Steam Web API):** ✓ Implemented. When `STEAM_API_KEY` is set, the backend (`internal/steam`) resolves each installed workshop map’s `preview_url` via `IPublishedFileService/GetDetails`, downloads the image once, and caches it on disk (`THUMB-NAIL_CACHE_DIR`, default `thumbnails/`). Thumbnails are served from `GET /api/maps/thumbnail/{workshopId}` and rendered on the workshop map cards, with a background prefetch kicked off when the workshop list loads. Concurrent requests for the same image are coalesced so Steam is hit at most once per item. When the key is unset (or Steam has no preview for an item), the card falls back to the gradient placeholder. `STEAM_API_KEY` is distinct from `STEAM_GSLT`.
 - **Map cycle editor:** Displays an ordered list of maps in the rotation (persisted in `localStorage` under `nokit_map_cycle`). Users can remove maps from the cycle; the default cycle includes 7 competitive maps. “Save mapcycle.txt” button is ready for backend integration to write the cycle to the server’s `mapcycle.txt` file.
 - **Responsive grid:** Map cards display in a 4-column grid (responsive: 1 col mobile, 2 col tablet, 3 col desktop, 4 col wide) with hover states and click-to-change functionality.
 - **TODO:** Standard-map-pool thumbnail images (still placeholder gradients — these are not workshop items so are not covered by the Steam API path), drag-and-drop reordering for map cycle.
-

6. CVAR Presets

Page title: **CVAR Presets**. Subtitle: 7 applied · changes preview as inline rcon · server.cfg untouched until "Persist".

Top-right buttons: + New custom preset, ↓ Export .cfg, green ⚡ Apply all selected.

Layout

Left ~2/3: preset categories, each a bordered group with a header CATEGORY N / M APPLIED and a row of toggle chips (checkbox pill; selected = green fill + checkmark). Each group ends with a + custom... chip to add a custom preset.

Right ~1/3: **Inline preview** panel + **Quick switch** panel.

Preset categories & chips

- **ROUND SETTINGS 2 / 6 APPLIED:** MR12 (24+0T) ✓ · MR15 (30) · Short (B09) · Warmup 15s ✓ · Warmup 60s · Freeze 10s · + custom...
- **ECONOMY 2 / 4 APPLIED:** Comp default ✓ · Pistol only · Rich start · 1:55 round ✓ · + custom...
- **MOVEMENT 1 / 3 APPLIED:** Default ✓ · Surf · KZ · + custom...
- **SERVER 2 / 4 APPLIED:** 128 tick ✓ · 64 tick · LAN · Pure 2 ✓ · + custom...
- **PRACTICE 0 / 4 APPLIED:** Nade practice · Infinite nades · No-flash bots · Auto-respawn · + custom...

Inline preview panel (right)

Header: >> **Inline preview** — right: 7 commands (count updates live). Monospace numbered listing grouped by preset comment header:

- #01 # MR12 (24+0T) → mp_maxrounds 24 ; mp_overtime_enable 1
- #02 # Warmup 15s → mp_warmuptime 15
- #03 # Comp default → mp_startmoney 800 ; mp_maxmoney 16000
- #04 # 1:55 round → mp_roundtime 1.92
- #05 # Default → sv_accelerate 5.5 ; sv_airaccelerate 12
- #06 # 128 tick → sv_minrate 128000 ; sv_maxupdaterate 128

Footer: ✓ = to apply & persist + green **Apply & persist** button.

Interaction (verified): toggling a preset chip live-updates the inline preview — enabling MR15 (30) inserted a #02 # MR15 (30) → mp_maxrounds 30 ; mp_overtime_enable 0 block and the command counter incremented 7 → 8.

Quick switch panel (right, below preview)

Header: ⚡ **Quick switch**. Full scenario presets (each with a keyboard shortcut badge):

- Match – 5v5 MR12 + 0T — green active badge
- Practice – solo nades — %1
- Pug – public retake — %2
- LAN – sv_lan 1, no auth — %3

Implementation notes: presets preview as inline RCON commands; server.cfg is untouched until the user hits **Persist**. Applying sends the commands over RCON; persisting writes them to the config.

7. Config Editor

Page title: **Config Editor**. Subtitle (breadcrumb path): `/home/cs2/server/csgo/cfg/gamemodes/gamemode_competitive_server.cfg`.

Top-right: amber `unsaved` badge, `↶ Reload from disk` button, `>_ exec via rcon` button, green `✓ Save & apply` button.

File browser (left panel, tree)

Header: `cfg /` with a `+` (new file) button. Tree entries show a “last modified” age on the right:

- `autoexec.cfg` — 2d
- `server.cfg` — 1h
- `banned_users.cfg` — 3h
- `banned_ip.cfg` — 7d
- **gamemodes** (folder, 4)
 - `gamemode_competitive.cfg` — 12m
 - `gamemode_competitive_server.cfg` — (open/selected)
 - `gamemode_casual.cfg`
 - `gamemode_deathmatch.cfg` — 14d
- **matchzy** (folder, 3)
 - `mr12_competitive.cfg` — 8h
 - `live.cfg` — 0h
 - `warmup.cfg` — 0h
- **cs_sharp** (folder, 2)
 - `WeaponPaints.json` — 5d
 - `MatchZy.json` — 8h

Editor (right panel — code editor with line numbers)

Top status strip (right): `utf-8 · LF · 80 lines · last edit 12m ago by m0use@`.

Syntax-highlighted `.cfg` content with line numbers. Comments in green (`// ...`), cvar names one color, values another. Sample content:

```

1 // gamemode_competitive_server.cfg
2 // matchzy + tournament defaults - fra1.mm
3 // last edit: m0use@ - 12:35
4
5 mp_maxrounds                24
6 mp_overtime_enable          1
7 mp_overtime_maxrounds       6
8 mp_overtime_startmoney      16000
9 mp_freezetime                15
10 mp_roundtime                1.92
11 mp_warmuptime               15
12 mp_startmoney               800
13 mp_maxmoney                 16000
14 mp_buy_anywhere             0
15 mp_team_intro_period        0
16
17 sv_pure                      2
18 sv_minrate                  128000
19 sv_maxupdaterate            128
20 sv_minupdaterate            128
21 sv_cheats                    0
22
23 // matchzy
24 matchzy_minimum_ready_required 5
25 matchzy_kick_when_no_match_loaded 0
26 matchzy_demo_path            "matchzy_demos"
27 matchzy_chat_prefix           "[>nokit<]"

```

Quirk: in the demo the editor content is static — selecting another file in the tree does not swap the shown buffer. In the real implementation, selecting a file loads its buffer; the `unsaved` badge indicates dirty state; `Save & apply` writes to disk and can `exec` via `rcon`.

Config Management — Implemented

 **Current implementation status (this fork):** The Config Editor is fully built and backed by a real config-management subsystem. It operates in **two runtime-selected modes** (mounted volume / panel fallback), mirroring the workshop-map two-mode design. The frontend lives in `web/src/pages/config-editor-page.tsx` (exported as `ConfigEditorPage`, rendered from `/servers/:id/config` in `App.tsx`); the backend is `internal/configs/` (`Manager`, `ConfigStore`), `internal/store/configs.go` (SQLite `panel_configs` table), and `internal/api/configs.go` (REST endpoints).

- **Mounted volume mode** — When the CS2 server's `cfg` directory is mounted into the panel container, the panel reads and writes `.cfg` files **directly from/to the CS2 server filesystem**. CRUD operations (list/get/save/delete) act on real files on disk.
- **Panel fallback mode** — When no config directory is mounted, the panel stores configs in its own **SQLite database** (`panel_configs` table, backed by `internal/store/configs.go`). Configs are authored, edited, and deleted entirely within the panel's DB with no filesystem access to the game server.
- **CodeMirror 6 editor** — The editor (`web/src/components/cfg-editor.tsx`) uses CodeMirror 6 with **CS2 `.cfg` syntax highlighting** (comments, commands/cvars, quoted strings, numbers via `web/src/lib/cs2-cfg-language.ts`) and **cvar autocomplete** for common CS2 console variables and commands.
- **Exec in mounted mode** — Running `exec` via RCON sends a single `exec <filename>` command over RCON, so the CS2 server loads the file straight from disk.

- **Exec in panel mode** — Since there's no file on the server to `exec`, Send Commands via RCON parses the config content and sends **each non-empty, non-comment line as an individual RCON command**, applying the settings live.
- **CONFIG_BASE environment variable** — Overrides the default `/configs` base directory the panel scans for per-server config mounts (`<CONFIG_BASE>/<serverId>`), allowing a custom mount path.
- **Auto-detection of writable mounts** — Whether a mounted config directory is writable is **auto-detected at runtime** via a write-probe (the panel creates and deletes a temp file in the mount) — it is not a config flag. A read-only (`:ro`) mount keeps the editor read-only; a read-write (`:rw`) mount enables saving and deleting files. The editor UI surfaces the active mode and writability.

The demo reference for the page (file-browser tree, breadcrumb, footer status strip) is described below; the implemented editor keeps the same UX shell — file browser sidebar, unsaved-changes badge, Reload/exec/Save toolbar — while swapping the mock buffer for real mode-aware loading and persistence.

8. Plugins

Page title: **Plugins**. Subtitle:

8 installed · 1 update available · 1 error · 1 warning · CS# 1.0.305 + Metamod 2.0.0-12 .

Top-right buttons: Reload all , Update all (1) , green + Browse registry .

Tabs (count badges)

Installed 8 · Browse 24+ · Error log 1 (Error log badge is red).

Installed tab

Controls: filter input `filter installed` — name, author, tag ; segmented status filter

[all | enabled | disabled | has update] (all selected); Install from .dll button (right).

Row layout: letter-avatar icon · name · version · status badge · tag badge(s) · author @handle · updated <age> · category · install path `/addons/cs_sharp/plugins/<x>` · description line · then on the right: an **enable/disable toggle switch** + Edit config + View logs + Details + red trash/delete icon.

Status badges: green `enabled` , grey `disabled` , amber `warn` , red `failed` , blue `core` , amber `update` → x.y.z .

Installed plugins:

Plugin	Ver	Status	Author	Updated	Category	Path	Description
CounterStrikeSharp	v1.0.305	enabled, core	@roflmuffin	12 May	core	/addons/cs_sharp/plugins/css	C# scripting host for CS2 dedicated servers. Loads .NET 8 plugin assemblies and exposes the gameserver event bus, RCON, and cvar APIs to managed code.
Metamod:Source	v2.0.0-12	enabled, core	@AlliedModders	08 May	core	/addons/cs_sharp/plugins/mm	Source engine plugin loader. Required by CounterStrikeSharp.
MatchZy	v0.8.7	enabled, update → 0.8.8	@shobhit-pathak	3d ago	competitive	/addons/cs_sharp/plugins/matchzy	Pug, scrim, and tournament match management with knife rounds, tactical pauses, and per-round demos.
KitsuneLab.MovementApi	v1.4.0	enabled	@kitsunelab	11d ago	utilities	/addons/cs_sharp/plugins/mov	Movement hook API used by surf, kz, and bhop plugins. Provides tick-accurate per-player movement callbacks.

WeaponPaints	v4.0	warn	@Nereziel	5d ago	fun	/addons/cs_sharp/plugins/paints	Skin / glove / knife chooser. Persists selection per SteamID via MySQL.
CS2-SimpleAdmin	v1.6.1	enabled	@daffyy	1mo ago	admin	/addons/cs_sharp/plugins/sa	In-game admin commands — kick, ban, gag, mute, freeze, slay. Compatible with SourceMod admins.cfg.
CallAdmin	v0.6.2	disabled	@rumblefrog	2mo ago	admin	/addons/cs_sharp/plugins/cadm	In-game report → Discord webhook with player context attached.
GotvFix	v0.3.0	failed	@whisper	14d ago	utilities	/addons/cs_sharp/plugins/gotv	TCP relay shim for GOTV — works around Valve's GOTV2 broken multicast on Linux 6.x kernels.

- The **failed** GotvFix row shows an inline red error box: [ERR] bind(0.0.0.0:27020) → EADDRINUSE — gotv relay disabled.

Browse tab (plugin registry / marketplace)

Left rail:

- **CATEGORIES:** All 24 · Admin tools 4 · Competitive 4 · Fun 4 · Utilities 4 · Maps 3 · Gamemodes 5.
- **FILTERS** (checkboxes): Has screenshots · CS2 compatible (checked) · MIT / Apache · No DB required.
- **SOURCES:** Registry 327 (active) · Custom git 3 · Local .dll.

Top bar: search search 327 plugins — name, author (with / regex); sort options [trending | most installed | recently updated | name] (trending selected).

FEATURED cards (icon · name · author · version · badge featured / installed · description · ↓ downloads · <age> · green install button):

- CS2-RetakesAllocator v2.1.0 @b3none · featured · “Full retakes implementation — bombsite rotation, automatic bomb planter, weapon allocator with...” · ↓47.4k · 3d ago · [install]
- MatchZy v0.8.7 @shobhit-pathak · installed · “Pug, scrim & tournament match management with knife round, pauses, demos.” · ↓92.1k · 3d ago
- WeaponPaints v4.0 @Nereziel · installed · “Skin / glove / knife chooser with web profile editor. MySQL-backed.” · ↓88.0k · 5d ago
- Surf Timer v1.0.4 @kitsunelab · featured · “Per-map records, leaderboards, and per-zone splits with a TAS-quality replay bot.” · ↓21.6k · 4d ago · [install]

ALL PLUGINS · 24 (row: icon · name · version · author · ↓ downloads · updated · category · tag chips · Install button + external-link icon):

- AccessChecker v1.1.0 @j00nfist · ↓24.6k · 5d ago · Admin tools · #groups #auth · “Steam group + flag-based access control. Verifies via Steam Web API on connect with 5-minute cache.”
- VoteManager v2.0.1 @riffrrri · ↓18.2k · 2w ago · Admin tools · #voting · “Veto, map vote, mute vote, kick vote. Configurable thresholds and player-side menus.”
- SpectatorTools v0.9.4 @b3none · ↓9.7k · 1mo ago · Admin tools · #debug · “Force spectate, freeze, no-clip, and god-mode admin tools.”

Error log tab

Controls: search search plugin output — message, plugin name, level ; segmented level filter [all | errors | warnings | info] ; ↓ Export button (right).

Monospace log lines YYYY-MM-DD HH:MM:SS [LEVEL] PluginName — message . Levels color-coded: [WARN] amber, [ERR] red, [INFO] grey/blue.

-

2026-05-21 12:48:02 [WARN] WeaponPaints — failed to fetch skin metadata for STEAM_1:1:43712001 (timeout 5s)

- 2026-05-21 12:18:54 [ERR] GotvFix — bind(0.0.0.0:27020) → EADDRINUSE — gotv relay disabled
- 2026-05-21 11:30:11 [INFO] MatchZy — matchzy: live config "mr12_competitive.cfg" applied
- 2026-05-21 11:08:31 [INFO] CounterStrikeSharp — plugin host started · 8 plugins loaded in

412ms

```
- 2026-05-21 10:55:14 [INFO] MatchZy - autoloader: warmup.cfg → live.cfg transition configured
- 2026-05-21 10:42:11 [INFO] WeaponPaints - mysql: connected to 127.0.0.1:3306 (db=weapons,
latency 1.4ms)
- 2026-05-21 10:39:02 [WARN] GotvFix - falling back to TCP relay - kernel multicast probe
failed
- 2026-05-21 10:38:54 [ERR ] GotvFix - unable to subscribe to GotvFix.RelayPort (27020) -
retrying in 30s
Footer: plugin-log } last 24h · 1 err · 2 warn · 12 info · 8 lines + clear button.
```

9. Scheduler

Page title: **Scheduler**. Subtitle: 4/5 active · system tz: Europe/Berlin · cron via nokit-relay .
 Top-right buttons: ⚡ Run now... , green + New task .

Jobs table (left, main)

Columns: NAME | SCHEDULE | COMMAND | LAST | NEXT . Each row has a status dot on the left (green = active, grey = disabled). NEXT run is bold when upcoming.

●	NAME	SCHEDULE (cron)	COMMAND	LAST	NEXT
green	Nightly restart	0 5 * * *	sv_shutdown	2026-05-21 05:00	2026-05-22 05:00
green	Warmup reset	*/30 * * * *	mp_warmup_start	12:30	13:00
green	Workshop sync	0 4 * * 0	ds_workshop_sync	2026-05-18 04:00	2026-05-25 04:00
green	Daily backup	0 3 * * *	exec ./backups/snap.sh	2026-05-21 03:00	2026-05-22 03:00
grey	Pug auto-shuffle	0 19 * * 5	mz_shuffle	2026-05-16 19:00	2026-05-23 19:00 (disabled)

RECENT RUNS table (below jobs)

Columns: WHEN | TASK | DURATION | RESULT . RESULT is a colored pill: green ok , amber N retries , grey disabled .

WHEN	TASK	DURATION	RESULT
12:30:00	Warmup reset	0.08s	ok
12:00:00	Warmup reset	0.07s	ok
05:00:00	Nightly restart	14.2s	ok
03:00:00	Daily backup	2m 04s	ok
04:00 (Sun)	Workshop sync	8m 41s	3 retries (amber)
19:00 (Fri)	Pug auto-shuffle	—	disabled (grey)

Presets panel (right)

Header: ✧ **Presets**. Clickable preset cards (name + cron/trigger + command, monospace grey):

- Nightly restart — 0 5 * * * → sv_shutdown
- Warmup reset every 30m — */30 * * * * → mp_warmup_start
- Pre-match script — on-event match → matchzy_load mr12
- GOTV demo cleanup — 0 6 * * 0 → find demos -mtime +30 -delete
- Discord ping if down — on-event down → webhook discord_alerts

Next 24h panel (right, below presets)

Header: **Next 24h**. Timeline list of upcoming runs (time + task):

- 13:00 Warmup reset
- 13:30 Warmup reset
- 14:00 Warmup reset
- — hourly until 04:30 — (collapsed range indicator)
- 05:00 Nightly restart
- 03:00 Daily backup

Implementation notes: cron scheduling runs via nokit-relay. Jobs support both cron expressions and on-event triggers (match, down). System timezone shown: Europe/Berlin.

10. Admin

Page title: **Admin**. Subtitle: 4 users · audit retention 90d · Discord webhook configured · GOTV port 27020.

Top-right buttons: ↓ Export audit log, green + Invite admin.

Tabs

Users & roles 4 · Audit log 10 · Integrations · GOTV & demos.

Users & roles tab

Users table columns: USER | EMAIL | ROLE | MFA | LAST SEEN. ROLE is a colored pill; MFA is green on / amber off pill.

USER	EMAIL	ROLE	MFA	LAST SEEN
m0use@	m@m0use.dev	owner (green)	on	12:51 — now
jaeger	p.j@axum.io	admin (blue)	on	2h ago
kira	kira@scrim.gg	admin (blue)	off (amber)	1d ago
ref-bot	discord://ref	moderator (grey)	on	4d ago

Role matrix panel (right) — Header **Role matrix**. Columns: CAPABILITY | OWNER | ADMIN | MOD (✓ = granted, · = not):

CAPABILITY	OWNER	ADMIN	MOD
rcon console	✓	✓	·
kick / ban	✓	✓	✓
edit configs	✓	✓	·
install plugins	✓	✓	·
manage admins	✓	·	·
manage servers	✓	·	·

Audit log tab

Controls: search search by user, action, target, time; time-range segmented filter

[today | 7d | 30d | all] (today selected); type filter [all | writes | signin] (writes selected).

Columns: WHEN | WHO | ACTION | TARGET + a status check (✓ ok / ⚠ warning icon at far right). ACTION is a colored keyword pill (cmd, kick, plugin, map, preset, config, ban, signin).

WHEN	WHO	ACTION	TARGET
-----	-----	-----	-----


```
| 12:51:08 | m0use@ | cmd | mp_warmup_end | ✓ |
| 12:49:51 | jaeger | kick | h4rdy_woods (STEAM_1:1:77124930) — afk | ✓ |
| 12:48:02 | system | plugin | WeaponPaints warn: fetch timeout | △ |
| 12:40:11 | m0use@ | map | de_mirage (was de_inferno) | ✓ |
| 12:38:00 | jaeger | preset | apply: MR12 (24+OT) | ✓ |
| 12:35:12 | m0use@ | config | edit gamemode_competitive.cfg (+4 -2) | ✓ |
| 12:30:00 | cron | cmd | mp_warmup_start | ✓ |
| 12:18:54 | system | plugin | GotvFix bind failure | △ |
| 12:14:33 | kira | ban | lobotomy — wallhack — 21d | ✓ |
| 12:02:11 | m0use@ | signin | from 84.124.221.9 — chrome/macOS | ✓ |
```

Integrations tab (4 cards)

Discord webhook — status ● connected (green):

- url `https://discord.com/api/webhooks/123•••••/k83a1••••••••••`
- channel `#nokit-fra1`
- events `match start · match end · server down · plugin error · ban`
- last `12:51 — match started`
- Buttons: `test message`, `edit`, `red disconnect`.

Steam workshop API — status ● key valid (green):

- key `A1F8••••••••••••••••E0`
- quota `4,221 / 100,000 calls (24h)`
- collection `ws://3142099912 — 16 maps`
- last sync `2026-05-18 04:08 · 8m 41s · 0 errors`
- Buttons: `rotate key`, `green sync now`.

Steam game-server login token — status ● active (green):

- app `730 (cs2)`
- token `F38C1B••••••••••••••••`
- label `nokit-fra1`
- added `2026-04-11`

Backup target — status ● S3 :

- bucket `s3://nokit-backups`
- prefix `fra1.mm/`
- retention `30d (cfg) · 14d (demos)`
- last `03:00 · 412 MB · ok`

GOTV & demos tab

Demo files table columns: FILE | SIZE | DURATION | DLS (downloads):

FILE	SIZE	DURATION	DLS
----- ----- ----- -----			
2026-05-21_de_mirage_mr12_team-0_vs_team-B.dem	94.2 MB	46m	2
2026-05-20_de_inferno_scrim_axum_vs_nbk.dem	102.8 MB	52m	0
2026-05-20_de_nuke_pug.dem	78.4 MB	38m	1
2026-05-19_de_anubis_eu-quals.dem	121.0 MB	1h 04m	8

GOTV settings panel (right) — Header **GOTV settings** (cvar → value):

- `tv_enable 1`
- `tv_port 27020`
- `tv_maxclients 32`

- `tv_delay 90s`
- `tv_autorecord 1`
- `tv_chatgroup "nokit-gotv"`
- `status ● 1 spectator · 0.4 mbps (green)`
- **DEMO RETENTION:** keep [14 days] then [upload to S3] (editable pills).

11. Multi-server switcher

Opened from the top-left server dropdown (`fra1.mm · 5.39.42.118:27015`).

Dropdown panel:

- Section label **SERVERS**. Each row: online status dot + name + `ip:port` · region + player-count badge + `×` remove button.

| Name | Address | Region | Players |

|-----|-----|-----|-----|

| fra1.mm (active) | 5.39.42.118:27015 | Frankfurt | 9/12 |

| seal.scrim | 10.0.1.22:27015 | Seattle | 10/10 |

| home.lan | 192.168.1.40:27015 | Local | 3/10 |

| practice-aws | 54.218.7.92:27015 | us-west-2 | 0/8 |

| retake-old | 88.99.10.4:27015 | Hetzner | — (offline, grey dot) |

- Section label **ADD SERVER** — inline form: `name` text input, `ip:port` text input, green **Add** button, and a `rcon password` input below (full-width).

- Clicking a server row switches the active server (updates top-bar tickrate/players/map and all pages).

Corresponds to CLI `srv > add [hostname] [IP:PORT]` .

12. Login / Auth & Session

The public demo (`nokit.app/demo`) is a pre-authenticated single-page mock — there is **no separate /login route** (visiting `/login` returns 404). Auth is documented rather than shown as a live form:

- **No SaaS accounts / no telemetry.** Self-hosted only.
- Initial admin set via env var in docker-compose: `NOKIT_ADMIN: m0use@local` .
- Authentication is via **SteamID or email**, with **MFA support** (see Admin → Users & roles: MFA `on / off` per user).
- **Role-based access control:** Owner / Admin / Moderator, enforced by the per-capability role matrix (see Admin).
- Sign-in events are recorded in the Audit log (e.g. `signin` — from `84.124.221.9` — `chrome/macOS`).
- Current session identity is shown in the top bar as `m0use@` with an avatar; a **power/logout icon** sits at the far right of the top bar. (In the demo these are static — the user menu does not open a drop-down.)
- Session/connection state surfaced globally: sidebar footer `● WS` indicator (note: docs describe live log transport as **SSE**, and RCON over loopback; the UI also shows a WS status pill) and footer `● srcds` `OK · ● relay OK` .

This fork's implementation note: Auth is currently password-based login with **session tokens persisted in SQLite** (see Feature Status Table §1).

13. Global search / Command palette

- Top-bar search field placeholder `cvars, players...` with a `⌘K` keyboard shortcut badge.
- Intended as a command palette to jump to `cvars/players/pages`. In the demo, activating it routes to the **RCON Console** (the `⌘K` shortcut and the sidebar RCON `⌘K` badge are linked). No floating overlay palette renders in the static demo.

Cross-cutting UI / Design notes (for re-implementation)

- **Theme:** dark UI (near-black background `#0d0d0f` -ish), green primary accent (buttons, active nav, online dots), monospace fonts for logs/console/cvars, sans-serif for chrome.
- **Status color language:**
 - **green** = ok / online / enabled / active
 - **amber/orange** = warning / T-side / update-available / temporary
 - **red** = error / failed / ban / danger / permanent
 - **grey** = disabled / offline
 - **blue** = CT-side / admin-role / core-plugin / info
- **Common layout pattern:** page header (title + descriptive subtitle on the left, action buttons on the right) → optional tab bar with count badges → main content (often a left main panel + right sidebar of secondary panels).
- **Segmented toggles** used widely (match/practice/pug, all/CT/T/spec, live/paused, time ranges).
- **Tables** use uppercase small-caps column headers, colored pill badges for status/role/team, and reveal inline row-action buttons on hover/selection.
- **Every mutating action** is designed to be logged to the Audit log with actor + action + target.
- **Refresh cadence:** dashboard/stats and player list auto-refresh every ~4s; logs stream via SSE.
- **Update frequency example:** page footer clock ticks live (observed 16:16 → 16:30 across the session), timezone `Europe/Berlin (UTC+2)`.
- **Version strings:** app `nokit v0.1.0-rc.1`, relay `v0.4.2`, CS# `1.0.305`, Metamod `2.0.0-12`, CS2 build `1.40.6.3` / protocol v15 build 9462.
- **Deploy target:** Docker image `ghcr.io/nokit/nokit:latest`, default web port `8080`, data volume `/data`, runs alongside `srcds` (loopback RCON). Single ~14 MB Go binary.

Bug Fixes & Stability Improvements

RCON Connection Stability (2026-07-11)

Problem: RCON connections would die during gameplay with a critical nil pointer dereference panic. The connection would close unexpectedly (due to network issues, server restarts, or CS2 timeouts), and the code would attempt to use the closed connection without proper validation, causing crashes.

Root Causes Identified:

1. **Nil pointer dereference** — After reconnection attempt, the code would call `client.Execute()` without checking if the client was successfully created
2. **Race condition** — Client was retrieved with a lock but used without holding the lock, allowing another goroutine to close it in between
3. **No reconnection backoff** — Rapid retry loops would hammer the server when connections failed

repeatedly

4. **Insufficient error context** — Errors didn't distinguish between different failure modes

Fixes Implemented (`internal/rcon/rcon.go`):

1. **Added nil check after reconnection** — Prevents panic by validating client exists before execute
2. **Hold lock during execute** — Prevents race condition where client could be closed between retrieval and execution
3. **Exponential backoff with tracking** — Added `lastFailTime` and `failCount` to connection struct; implements exponential backoff (1s → 2s → 4s → 8s → 16s → 30s max) to prevent reconnection storms
4. **Better error messages** — More descriptive errors with context about failure mode and backoff state
5. **Connection state reset** — Properly reset failure counters on successful reconnection

Technical Details:

- Backoff calculation: `baseDelay * (2^(failCount-1))` capped at 30 seconds
- Failure tracking persists across attempts until successful connection
- Dial timeout: 5 seconds, command deadline: 10 seconds
- Lock held during final execute to ensure atomicity

Impact: Eliminates crashes during network instability, server restarts, or CS2 RCON timeouts. Reduces server load by preventing rapid reconnection attempts.

This document was compiled from observations of the nokit demo. Keep it updated as features move from  (demo/planned) to  (implemented).